MyCodeSchool

Cifre: exerciții în două stiluri

Partea 1 · David · exerciții

Fiecare problemă se rezolvă de două ori, în `cifre.py`:
Stil A — ca în C++: numai cu `while`, `if`, `%`, `//` și funcțiile scrise de tine. Fără `str`, `sum`, `max`, `min`, `len`, `count`, `replace`, `join`, `gcd`.
Stil B — stil Python: funcția cu același nume + `_py`, cu numărul transformat în text prin `str(n)` și uneltele din teorie, secțiunea 4.
Ambele funcții întorc rezultatul cu `return` și nu citesc, nu afișează nimic. Numerele sunt naturale, cel mult 1 000 000 000.

Înainte de problema 1 — proiectul tău

În același folder: `cifre.py` (funcțiile) și `main.py` (citirea și afișarea). În `main.py` scrii `from cifre import (...)` cu numele ambelor funcții ale problemei, iar codul din „În main()" îl pui în `def main() -> None:`. Ce afișează programul trebuie să fie identic cu „Ieșire așteptată": aceeași valoare de două ori, o dată din fiecare stil.

cifre.py

```
def suma_cifrelor(n: int) -> int:
    ...


def suma_cifrelor_py(n: int) -> int:
    ...
```

main.py

```
from cifre import (
    suma_cifrelor,
    suma_cifrelor_py,
)


def main() -> None:
    n = int(input())
    print(suma_cifrelor(n), suma_cifrelor_py(n))


if __name__ == "__main__":
    main()
```

La clasă

Sintaxa nouă, pe algoritmi pe care îi știi.

1. Suma cifrelor

În `cifre.py`: `def suma_cifrelor(n: int) -> int:` (stil A) și `def suma_cifrelor_py(n: int) -> int:` (stil B)
Întoarce suma cifrelor lui `n`.

În main()

```
n = int(input())
print(suma_cifrelor(n), suma_cifrelor_py(n))
```

Tastezi

```
4738
```

Ieșire așteptată

```
22 22
```

2. Numărul de cifre

În `cifre.py`: `def numar_cifre(n: int) -> int:` (stil A) și `def numar_cifre_py(n: int) -> int:` (stil B)
Întoarce câte cifre are `n`. Pentru `n = 0` răspunsul e 1.

În main()

```
n = int(input())
print(numar_cifre(n), numar_cifre_py(n))
```

Tastezi

```
90210
```

Ieșire așteptată

```
5 5
```

3. Oglinditul și palindromul

În `cifre.py`: `def oglindit(n: int) -> int:` (stil A) și `def oglindit_py(n: int) -> int:` (stil B)

Întoarce numărul cu cifrele lui `n` în ordine inversă (zerourile de la început dispar: 1230 → 321). Scrii și `este_palindrom(n: int) -> bool` / `este_palindrom_py`: `n` se citește la fel de la ambele capete.

În main()

```
n = int(input())
print(oglindit(n), oglindit_py(n))
print(este_palindrom(n), este_palindrom_py(n))
```

Tastezi

```
1230
```

Ieșire așteptată

```
321 321
False False
```

4. Cifra maximă și de câte ori apare

În `cifre.py`: `def cifra_maxima(n: int) -> tuple[int, int]:` (stil A) și `def cifra_maxima_py(n: int) -> tuple[int, int]:` (stil B)

Întoarce, deodată, cifra maximă a lui `n` și numărul ei de apariții.

În main()

```
n = int(input())
m, k = cifra_maxima(n)
print(m, k)
m, k = cifra_maxima_py(n)
print(m, k)
```

Tastezi

```
39893
```

Ieșire așteptată

```
9 2
9 2
```

De exersat

Probleme de bac, în ambele stiluri.

5. Doar cifrele pare

În `cifre.py`: `def cifre_pare(n: int) -> int:` (stil A) și `def cifre_pare_py(n: int) -> int:` (stil B)

Pentru $n \geq 1$, întoarce numărul format din cifrele pare ale lui `n`, în ordinea în care apar. Dacă `n` nu are cifre pare, întoarce -1.

În main()

```
n = int(input())
print(cifre_pare(n), cifre_pare_py(n))
```

Tastezi

```
2738465
```

Ieșire așteptată

```
2846 2846
```

6. Ștergerea unei cifre

În `cifre.py`: `def sterge_cifra(n: int, c: int) -> int:` (stil A) și `def sterge_cifra_py(n: int, c: int) -> int:` (stil B)

Întoarce numărul obținut din `n` după ștergerea tuturor aparițiilor cifrei `c`. Dacă nu rămâne nicio cifră, întoarce -1.

În main()

```
n = int(input())
c = int(input())
print(sterge_cifra(n, c), sterge_cifra_py(n, c))
```

Tastezi

```
1213415
1
```

Ieșire așteptată

```
2345 2345
```

7. Prim în ambele sensuri

În `cifre.py`: `def prim_si_oglindit(n: int) -> bool:` (stil A) și `def prim_si_oglindit_py(n: int) -> bool:` (stil B)

Scrii întâi `este_prim(n: int) -> bool` / `este_prim_py`. Apoi `prim_si_oglindit` întoarce `True` dacă și `n`, și oglinditul lui sunt prime (13 și 31 da; 23 nu, 32 nu e prim).

În main()

```
n = int(input())
print(prim_si_oglindit(n), prim_si_oglindit_py(n))
```

Tastezi

```
107
```

Ieșire așteptată

```
True True
```

8. Fracția ireductibilă

În `cifre.py`: `def simplifica(a: int, b: int) -> tuple[int, int]:` (stil A) și `def simplifica_py(a: int, b: int) -> tuple[int, int]:` (stil B)

Pentru `a, b ≥ 1`, întoarce numărătorul și numitorul fracției `a/b` simplificate. În stilul A scrii și `cmmdc(a: int, b: int) -> int` cu algoritmul lui Euclid; în stilul B folosești `gcd`.

În main()

```
a = int(input())
b = int(input())
x, y = simplifica(a, b)
print(x, y)
x, y = simplifica_py(a, b)
print(x, y)
```

Tastezi

```
84
36
```

Ieșire așteptată

```
7 3
7 3
```

Provocări

Fără indicații în plus.

9. Număr perfect

În `cifre.py`: `def este_perfect(n: int) -> bool:` (stil A) și `def este_perfect_py(n: int) -> bool:` (stil B)

Un număr e perfect dacă e egal cu suma divizorilor lui mai mici decât el (28 = 1 + 2 + 4 + 7 + 14). În stilul A cauți divizorii doar până la radical, în perechi `d` și `n // d`.

În main()

```
n = int(input())
print(este_perfect(n), este_perfect_py(n))
```

Tastezi

```
496
```

Ieșire așteptată

```
True True
```

10. Cifra de control

În `cifre.py`: `def cifra_de_control(n: int) -> int:` (stil A) și `def cifra_de_control_py(n: int) -> int:` (stil B)

Aduni cifrele lui `n`, apoi cifrele rezultatului, și tot așa, până rămâne o singură cifră (98765 → 35 → 8). Întoarce acea cifră.

În main()

```
n = int(input())
print(cifra_de_control(n), cifra_de_control_py(n))
```

Tastezi

```
98765
```

Ieșire așteptată

```
8 8
```

11. Cifrele impare dublate

În `cifre.py`: `def dubleaza_impare(n: int) -> int:` (stil A) și `def dubleaza_impare_py(n: int) -> int:` (stil B)

Întoarce numărul obținut din `n` scriind fiecare cifră impară de două ori, pe loc (2357 → 2335577).

În main()

```
n = int(input())
print(dubleaza_impare(n), dubleaza_impare_py(n))
```

Tastezi

```
2357
```

Ieșire așteptată

```
2335577 2335577
```